



MEDIACREST
TRAINING COLLEGE

Advanced Data Analytics & Machine Learning

Advanced Data Auditing & Missingness Mechanisms – Week 1,
Day 2

Course Trainer: Victor Wandera Lumumba

Statistician, Data Analyst & BI Specialist

Mediacrest Training College

11 August 2026

Abstract

This session turns a messy business dataset (`Business_Data_Quality_Issues.csv`) into an **analysis-ready, model-ready table** — the foundation of every credible ML project.

- **Why auditing first:** data preparation consumes $\sim 80\%$ of analyst time; “garbage in, garbage out” is a policy risk, not just a technical one.
- **Missingness mechanisms:** MCAR vs MAR vs MNAR — and why each demands a different treatment.
- **Profiling & EDA:** automated `ProfileReport` plus manual `shape`, `dtype`, and missing-value checks.
- **Cleaning mechanics:** duplicates, median/Unknown imputation, and categorical standardisation in `pandas`.
- **Hands-on:** audit a national health dataset and document the bias introduced by missing clinical records.

Philosophy: *audit before you model* — every imputation is a modelling decision.

Session Objectives

- Understand and classify missingness mechanisms — **MCAR, MAR, MNAR** — and match each to a safe treatment.
- Conduct an **advanced data audit**: automated profiling plus manual checks on shape, dtypes, and missingness.
- Standardise **column names** and **categorical values** to eliminate silent join and grouping errors.
- Apply principled **missing-value treatment**: median for numeric, “Unknown” for categorical.
- Identify **bias introduced by missing clinical records** in a Kenyan health dataset and document it for policy-grade reporting.

Agenda

- 1 **Part 1: Why Data Auditing Matters** (the 80% rule, missingness mechanisms MCAR/MAR/MNAR)
- 2 **Part 2: Profiling & EDA Toolkit** (install, ProfileReport, shape/dtypes, missing checks)
- 3 **Part 3: Duplicates & Missing Treatment** (detect/drop dupes, median & Unknown imputation)
- 4 **Part 4: Standardising Values** (Gender, Region, ProductCategory, whitespace control)
- 5 **Part 5: Practical Application** (MOH-style health dataset, bias identification, deliverable)

Tooling

Everything runs in **Python** (pandas, NumPy, data-profiling) inside a Jupyter notebook. Source files: `Business_Data_Quality_Issues.csv` and, for the practical, a national health dataset.

Part 1: Why Data Auditing Matters

"Garbage in, garbage out — audit before you model."

The 80% Rule: Preparation Is the Work

Where analyst time really goes

- Data collection, cleaning & prep: ~80%
- Modelling & evaluation: ~20%
- Models inherit every flaw you skip.

Policy Risk

A biased or leaky dataset silently biases every recommendation you give to MOH, CBK, KRA, or a County Government.

Common data-quality defects

Defect	Consequence
Missing values	Biased estimates
Duplicates	Inflated weights
Inconsistent labels	Broken joins
Wrong dtypes	Silent NaN
Out-of-range values	Skewed models

Missingness Mechanisms: The Decision Framework

Before you impute, classify *why* data is missing. The mechanism determines the safe treatment.

Mechanism	Meaning	Detection clue	Safe treatment
MCAR Missing Completely At Random	Missingness unrelated to any observed or unobserved variable	Little's MCAR test; missingness evenly spread	Deletion or simple imputation
MAR Missing At Random	Missingness explained by <i>observed</i> variables	Missing rate varies by a known group	Model-based imputation (KNN, MICE)
MNAR Missing Not At Random	Missingness depends on the <i>unobserved</i> value itself	Pattern correlates with the value	Requires domain model; flag it

Rule of Thumb

Deletion is only safe under MCAR. Most real datasets are MAR or MNAR — impute conditionally or model the missingness.

MCAR: Missing Completely At Random

Definition

The probability a value is missing is *independent* of everything — observed and unobserved.

Detection

- Missingness looks like pure noise.
- Little's MCAR test ($p > 0.05$ suggests MCAR).
- Compare means/distributions of observed data across missing vs present groups.

Example

A lab machine jams randomly for an afternoon; samples processed in that window are missing regardless of patient health. Deleting them does not bias the analysis.

Warning

True MCAR is rare. Treat any convenient deletion assumption with suspicion.

MAR: Missing At Random

Definition

Missingness depends on *other observed* variables, but not on the missing value itself.

Detection

- Missing rate differs across known groups (county, gender, facility tier).
- Can be modelled using observed predictors.

Treatment

Conditional imputation: KNN, MICE, or group-wise medians.

Example

Blood-pressure readings are missing more often in *rural* clinics that lack automated cuffs. The missingness is explained by location (observed) — impute conditionally on county.

Key Insight

“At Random” is a misleading name: MAR is *systematic*, but the system is observable.

MNAR: Missing Not At Random

Definition

The probability of missingness depends on the *unobserved value itself*.

Detection

- Hardest to prove statistically.
- Requires domain reasoning and pattern inspection.

Treatment

Model the missingness explicitly, use sensitivity analysis, and **always flag it** in your report.

Example

Sicker patients skip clinic visits, so their clinical records are missing *because* they are unwell. Imputing their vitals with healthy-patient means understates disease burden.

Policy Danger

MNAR in a national health dataset can hide the very population a policy is meant to serve.

Part 2: Profiling & EDA Toolkit

"Automate the audit, then verify by hand."

Step 1: Install the Data Profiling Library

A profiling library generates an interactive HTML report — distributions, missingness, correlations, and alerts — in one command.

```
1 #!pip install fg-data-profiling
2 #!pip install ydata-profiling
```

Listing 1: Install the profiling package (run once)

Why Profile First?

It surfaces the defects from Slide 6 in seconds and gives you a baseline report to compare *before* and *after* cleaning.

Note

Use the ! prefix only inside Jupyter. In a terminal, drop it. Prefer ydata-profiling if fg-data-profiling is unavailable.

Step 2: Load the Required Libraries

```
1 import pandas as pd
2 import webbrowser
3 import numpy as np
4 import data_profiling
5 from data_profiling import ProfileReport
```

Listing 2: Standard imports for the audit

Role of each import

Library	Purpose
pandas	Load, inspect, transform data
webbrowser	Open the HTML report
numpy	Numeric ops, <code>np.nan</code>
ProfileReport	Generate the audit report

ModuleNotFoundError?

If `data_profiling` fails to import, install the package (Slide 12) and **restart the kernel**.

Steps 3–4: Load the Data & Generate the Profile

```
1 df = pd.read_csv("Business_Data_Quality_Issues.csv")
2 df.head()
3
4 profile = ProfileReport(df, title="Profiling Report",
5                           explorative=True)
6 profile.to_file('report.html')
7 webbrowser.open("report.html")
```

Listing 3: Load and profile the raw data

What to Look for in the Report

- **Overview:** row/column counts, memory, duplicate rate.
- **Variables:** per-column type, missing %, distinct count, distribution.
- **Alerts:** constant, highly-correlated, or skewed columns flagged automatically.

Step 5: Exploratory Data Analysis — Shape & Types

```
1 df.shape      # (rows, columns)
2 df.dtypes     # data type of every column
```

Listing 4: First manual checks

Why It Matters

shape confirms record volume; dtypes reveal columns that *should* be numeric or datetime but loaded as object — a classic source of silent NaN.

Red Flag

A Revenue column typed as object means currency symbols or commas are present — it needs string cleaning before any numeric imputation.

Step 5: Quantify Missingness

```
1 df.isnull().sum()           # raw counts
2 df.isnull().mean() * 100    # percentages
```

Listing 5: Missing counts and percentages

Interpret Against the Framework

Percentages alone do not tell the story.
Cross-tabulate missingness by group (county, facility, date) to test whether the pattern is MCAR, MAR, or MNAR.

Threshold Discipline

Columns missing >70% rarely survive as features.
Decide a policy up front and document every column you drop.

Step 6: Standardise the Column Names

```
1 df.columns = (  
2     df.columns  
3     .str.strip()           # remove leading/trailing spaces  
4     .str.replace(' ', '_') # spaces -> underscores  
5     .str.replace('(', '')  # drop parentheses  
6     .str.replace(')', '')  
7 )  
8 df.head()
```

Listing 6: Clean, consistent column names

Why Standardise Names?

Uniform names make filtering, joining, and model-building reliable and reduce typo-driven errors. Clean names are **cheap insurance** for every downstream step — especially when merging MOH, CBK, or county tables.

Part 3: Duplicates & Missing Treatment

"Remove the noise, impute with intent."

Step 7: Detect Duplicate Rows

```
1 dupes = df.duplicated()
2 dupes.sum()
3
4 # Print the duplicated rows for inspection
5 df[df.duplicated(keep=False)] \
6     .sort_values(by=list(df.columns)).head(10)
```

Listing 7: Count and inspect duplicates

Inspect Before Deleting

`keep=False` marks *all* copies so you can see the full duplicate group and confirm they are true repeats, not legitimate near-duplicates.

Duplicates Bias Models

Repeated rows overweight certain patients or transactions, distorting any aggregate or trained model.

Step 7: Drop the Duplicate Rows

```
1 df.drop_duplicates(keep='first', inplace=True)
2 df.shape
```

Listing 8: Remove duplicates, keep first occurrence

Verify the Drop

Compare `df.shape` before and after. The row count reduction should match the duplicate count from Slide 19.

Watch the Key

If the dataset has a natural key (patient ID, transaction ID), deduplicate *on that key*, not on all columns.

Step 7: Impute Numeric Columns with the Median

```
1 df = df.copy()
2
3 numeric_columns = df.select_dtypes(include=np.number).columns
4 for col in numeric_columns:
5     df[col] = df[col].fillna(df[col].median())
```

Listing 9: Median imputation for numeric features

Why the Median?

The median is robust to outliers, so a few extreme values will not drag the fill value. Reserve mean imputation for genuinely symmetric, outlier-free columns.

Link to Session 1: this is a simple, defensible treatment — appropriate when missingness is MCAR or when the missing share is small.

Step 7: Impute Categorical Columns with “Unknown”

```
1 categorical_columns = df.select_dtypes(  
2     include=['object', 'category']).columns  
3 for col in categorical_columns:  
4     df[col] = df[col].fillna("Unknown")
```

Listing 10: Unknown imputation for categorical features

Why "Unknown" and Not the Mode?

Filling with the most frequent category injects false signal. “Unknown” preserves the fact of missingness as a legitimate category — often itself informative (e.g., undocumented facility tier).

MNAR note: if records are missing *because* of the underlying condition, “Unknown” keeps that signal visible for later investigation.

Verify: Confirm No Missingness Remains

```
1 print(df.isnull().sum())  
2 df.head()
```

Listing 11: Display remaining missing values and preview

Every Imputation Is Logged

Before moving on, you should be able to state: which columns were imputed, with what value, and under which missingness assumption.

Empty Strings $\neq$ NaN

`isnull()` misses "". Always check for blank strings separately — a recurring theme in later cleaning steps.

Part 4: Standardising Values

"Same meaning, same spelling — everywhere."

Standardise Categorical Values: Gender

```
1 df['Gender'].value_counts()  
2  
3 df["Gender"] = df["Gender"].replace({  
4     "M": "Male",  
5     "male": "Male",  
6     "F": "Female",  
7     "FEMALE": "Female"  
8 })  
9 print(df["Gender"].value_counts())
```

Listing 12: Map Gender variants to canonical labels

Why It Matters

M, male, Male are three categories to a model. A `replace` dictionary collapses them into two clean labels — essential before any encoding step (Week 1, Session 2).

Standardise Categorical Values: Region

```
1 df['Region'].value_counts()  
2  
3 df["Region"] = df["Region"].replace({  
4     "NORTH": "North",  
5     "SOUTH": "South",  
6     "EAST": "East",  
7     "WEST": "West",  
8     "CENTRAL": "Central"  
9 })  
10 print(df["Region"].value_counts())
```

Listing 13: Normalise Region casing

Kenyan Context

The same discipline applies to the **47 counties**: enforce an official spelling list so “Nakuru”, “NAKURU”, and “Nakuru County” never become three separate groups.

Standardise Categorical Values: ProductCategory

```
1 df["ProductCategory"] = (  
2     df["ProductCategory"]  
3     .str.strip()  
4     .str.replace(r"\s+", " ", regex=True)  
5 )  
6 print(df["ProductCategory"].value_counts())
```

Listing 14: Strip and collapse whitespace

The Invisible Bug

"Electronics" and "Electronics " are different strings. `.str.strip()` removes edge spaces; `\s+` collapses internal double-spaces. Always re-run `value_counts()` to confirm the categories merged.

Part 5: Practical Application

"From a business CSV to a national health dataset."

Continuous Variables: Guard Against Invalid Ranges

```
1 summary = df[['Quantity', 'UnitPrice', 'Discount',  
2              'Rating', 'Revenue']].agg(  
3      ['count', 'mean', 'median', 'std', 'var', 'min',  
4      'max', 'skew', 'kurt']).T  
5 summary  
6  
7 # Replace negative quantities with NaN, then impute  
8 df.loc[df["Quantity"] < 0, "Quantity"] = np.nan  
9 df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
```

Listing 15: Full summary statistics, then fix negatives

Range Checks Are Domain Rules

skew and kurt flag distribution problems; negative counts flag impossible values. Convert impossibilities to NaN *first*, then impute — never impute over garbage.

Rating & Revenue: Treat Anomalies Consistently

```
1 # Negative ratings -> NaN -> median
2 df.loc[df["Rating"] < 0, "Rating"] = np.nan
3 df["Rating"] = df["Rating"].fillna(df["Rating"].median())
4
5 # Negative revenue -> NaN -> median
6 df.loc[df["Revenue"] < 0, "Revenue"] = np.nan
7 df["Revenue"] = df["Revenue"].fillna(df["Revenue"].median())
```

Listing 16: Fix out-of-range Rating and Revenue

Consistent Policy

The same three-step pattern — **detect** → **nullify** → **impute** — applies to every bounded column (Rating 0–5, Revenue ≥ 0 , vitals within physiological range).

Practical: Prepare a National Health Dataset

Scenario: The Ministry of Health hands you a national patient dataset for a predictive model on hypertension risk. Clinical records are unevenly complete across counties.

Your Mission (apply today's toolkit):

- 1 **Profile** the dataset with ProfileReport and record every quality alert.
- 2 **Quantify** missingness per column and **classify** the mechanism — MCAR, MAR, or MNAR — for the three worst columns.
- 3 **Standardise** facility names, county labels (47 counties), and categorical codes.
- 4 **Treat** missing values: median for vitals, “Unknown” for facility/cadre categories.
- 5 **Document** the bias introduced by missing clinical records.

Hint: Cross-tabulate missingness by county to test for MAR/MNAR before choosing a treatment.

Practical: Identifying Bias from Missing Clinical Records

Questions to answer

- Are records missing more in certain counties or facility tiers?
- Do sicker patients have *more* missing vitals (MNAR)?
- Would median imputation pull rural/coastal averages toward the national mean?

Deliverable

A short memo: which columns, which mechanism, which treatment, and the residual risk.

Expected Finding

If missing vitals cluster in under-resourced counties, naive imputation *understates* their disease burden — the model then under-prioritises the very facilities that need support.

Policy-Grade Standard

Every imputation decision must be traceable, justified, and flagged for the model team.

Best Practices: Audit & Cleaning Hygiene

- **Profile twice:** once before cleaning, once after — the diff is your evidence of work.
- **Classify before you impute:** MCAR/MAR/MNAR determines whether deletion, median, or model-based imputation is safe.
- **Convert, then impute:** turn impossible or non-numeric values into NaN *before* filling.
- **Standardise relentlessly:** column names and categorical labels must be canonical before any join or encoding.
- **Document every decision:** what was dropped, what was filled, and why — this is your audit trail for MOH, CBK, or county reviewers.
- **Beware empty strings:** `isnull()` misses `""`; always check both.

Session Deliverables & What Comes Next

Today's Deliverable (due end of session)

- Completed **data audit report** on the health dataset (before/after profiling).
- Documented **missingness classification** and chosen treatment strategy per column.

Coming Up in Week 1

- **Session 2:** Outlier detection (IQR, Z-score), feature encoding, and scaling/normalisation.
- **Session 3:** Data leakage prevention and building a reproducible `sklearn` preprocessing pipeline.

Questions?

Let's discuss MCAR vs MAR detection, when median imputation misleads, or how to document bias for a policy audience.

Victor Wandera Lumumba

Statistician | Data Analyst | BI Specialist

Mediacrest Training College – Nairobi, Kenya

Advanced Data Analytics & ML | Week 1, Session 1 | 10 August 2026